

# Mini Transport Tycoon

During the semester, you have to implement a simplified transportation-economic simulation game similar to [Transport Tycoon](#). The goal of the game is for the player to organize road freight and passenger transport between cities and industrial facilities, by purchasing vehicles and establishing routes, in order to maximize profit.



*Képernyőkép az OpenTTD játékból*

An open-source community implementation of the sample game is freely available for trial:

<https://www.openttd.org/>

<https://store.steampowered.com/app/1536610/OpenTTD/>

## A Brief Description of the Game

The game takes place on a 2-dimensional top-down map featuring cities, industrial facilities, and a predefined road network connecting them. The player can build roads, place stops on the road network, purchase road vehicles, define routes, and use them to transport goods and/or passengers. The game runs in real-time, but the speed of the game can be accelerated.

### Map

The game operates on a grid-based map. The map contains pre-placed **cities** and **industrial facilities** (e.g., mine, farm, factory). Cities and industrial facilities have fixed positions and cannot be moved. A city or an industrial facility occupies multiple tiles on the map:

- A facility covers at least a 2×2 tile area,
- A city covers at least a 3×3 tile area.

Roads cannot be built on the tiles occupied by cities and industrial facilities; however, stops and roads can be connected along their edges. Within a city's area, there is a predefined internal road network modeling intra-city traffic, which the player cannot modify, but vehicles can use for travel.

### **Road construction**

Building road infrastructure is part of the core task. The player can build roads on empty tiles for a specified cost. Vehicles can only travel on the road network between industrial facilities and cities.

### **Goods and Demands**

The game must include at least 3 different types of transportable industrial raw materials/products (e.g., wood, paper, iron, steel), as well as passengers. Industrial facilities produce certain goods (e.g., a mine can produce iron ore) and consume others (e.g., a steel mill requires iron ore and produces steel). Cities demand passengers and/or finished products.

Demands change over time, but not instantly (slow increase or decrease).

### **Vehicles**

Only road vehicles are included in the task. There should be distinct transport vehicles for different raw material/product types, as well as buses for passenger transport. There must be at least 2 different vehicle types per category, with varying speeds, capacities, and maintenance costs.

Vehicles travel between stops, following a given route, and load if there is available cargo or passengers. Only one vehicle traveling in a given direction can occupy a road tile at a time. (Therefore, a maximum of 2 vehicles can be on one tile in total.)

### **Stops and Routes**

The player can place stops along roads, near cities and industrial facilities. Stops occupy one tile.

Circular routes can be assigned to vehicles (e.g.,  $A \rightarrow B \rightarrow C \rightarrow A$ ). A stop can belong to multiple routes. Vehicles automatically repeat their assigned route.

### **Economy**

The player starts with a given initial capital. Income is generated from successfully delivered goods or passengers. However, there are costs for building roads, maintaining vehicles, and purchasing new vehicles.

If the player's capital becomes negative, they go bankrupt and the game ends.

### **Time management**

The game runs in continuous time. There must be at least three time speed settings:

- pause
- normal
- fast (e.g. 2x speed)
- very fast (e.g. 4x speed)

## Display

A 2D top-down display is expected for the core task, where the content of individual tiles is illustrated by images. The map should visually show the industrial facilities, cities, the road network, vehicle positions, stops, and routes.

---

## Sub-tasks

The complexity of the base game is **2 units**. (1.5 units for teams of 4, 3 units for teams of 2). Each team must select enough sub-tasks from the list below during the task specification **so that the project's total complexity reaches 5 units**. Additional sub-tasks based on your own ideas can also be implemented, subject to consultation with the lab instructor.

The complexity value of the actually implemented functionality by the end of the semester serves as the upper limit for the final grade for every member of the team.

### Forests [0.5 complexity]

Trees can appear on empty tiles. There can be 1-4 trees on a tile. Over time, the number of trees on a tile can increase (e.g., 1 → 4), and new trees can appear on adjacent empty tiles.

Roads can also be built on forested tiles, but at a higher cost (clearing).

### Rivers and Lakes [0.5 complexity]

The map should also have water fields that, when initially placed (either pre-set or randomly generated), form lakes and rivers. There should be at least 3 different bridge types in the game, with different costs, maximum bridge distances, and speed limits.

### Terrain [1 complexity]

The terrain is not flat; tiles have varying heights, which should be visually represented using different slope-type tiles. Roads can only be built between tiles with the same height or a height difference of  $\pm 1$ . Vehicles should move slower on uphill slopes.

The player should have the option to terraform the terrain for a cost, i.e. change the height of empty spaces by 1. (But the absolute height difference between adjacent spaces should still be at most 2.)

### Garage [0.5 complexity]

The player should be able to build one or more garages, which also need to be connected to the road network. Vehicles can be purchased and maintained in the garage. Vehicles should automatically return to the garage for maintenance at specified intervals. The older a vehicle is, the more frequent maintenance it should require. There should be an option to sell over-aged vehicles to avoid further operational difficulties and costs.

**Overtaking vehicles [0.5 complexity]**

According to the basic task, vehicles cannot overtake and only one vehicle can be on a field in one direction. This subtask removes this restriction: if a faster vehicle catches up with a slower one and the oncoming lane (oncoming traffic in the given space) is empty, the vehicle can start overtaking.

During the overtaking, the vehicle can temporarily use the oncoming lane, but only if there is no oncoming traffic within a visible distance. If the overtaking cannot be completed safely, the vehicle must stay behind the slower one. The overtaking process must also be clearly visible (e.g. the vehicle shifts out of its lane).

**Traffic Lights [1 complexity]**

In the basic mission, the "first come, first served" principle applies at intersections. This expansion allows for complex traffic control by allowing the player to install traffic lights at intersections (junctions 3 or 4) of their own built road network.

The lights switch between different directions at fixed intervals. By clicking on the light, the player can set the length of the green signal per direction in an interface (e.g. the main road should get a longer green).

Vehicles are required to stop at red signals. The current color of the lights (at least red / green) should be clearly displayed on the map at the junctions.

**City Growth [0.5 complexity]**

Cities should grow in size over time. City growth should be faster if they have regular goods and/or passenger traffic. As cities grow, they should expand into adjacent fields (where the player has not built), creating new buildings and a road network. The continuity of the city's internal road network should be maintained.

**Minimap [0.5 complexity]**

The game map should be larger than the displayed area and should be scrollable in both X and Y dimensions for navigation. For easier orientation, a navigable minimap should be part of the game interface.

**Persistence [0.5 complexity]**

It should be possible to save a given game state and later load a selected saved game state to continue playing. After loading, vehicles that were in motion at the time of saving should resume their journey from where they were located. The system should also support handling multiple save files.

**Continuous movement [0.5 complexity]**

The movement of vehicles between tiles on the game map should not be jumpy but continuous. Vehicles can still logically be exclusively on a single tile at any time, but their movement between tiles should be animated and smooth.

**Map Generation [0.5 complexity]**

The initial map should not be fixed or completely randomly generated. Instead, the application should generate it using some map generation algorithm, based on logical rules. A procedural algorithm can be chosen (possible examples: *Wave Function Collapse* algorithm, *Perlin Noise* algorithm, *Simplex Noise* algorithm), or even an AI-based approach.

**2.5D Graphics [0.5 complexity]**

The requirement for the base task is to implement a top-down graphic display where every object appears within its own tile. This sub-task involves implementing a 2.5-dimensional graphic style where objects can visually overflow beyond their own cell's boundaries. *Cannot be chosen together with the 3D Graphics sub-task.*

**3D Graphics [1 complexity]**

The requirement for the base task is to implement a top-down graphic display where every object appears within its own tile. This sub-task involves implementing a true, rotatable 3-dimensional display. *Cannot be chosen together with the 2.5D Graphics sub-task.*